

DATA PREPARATION



CONTENTS

- What is data preparation?
- Why is data preparation important?
- Data preparation steps
- Data Cleaning
- Handling Missing Value
- Handling Outliers
- Feature Scaling
- Encoding
- Data preparation: Myth Vs Reality

WHAT IS DATA PREPARATION?

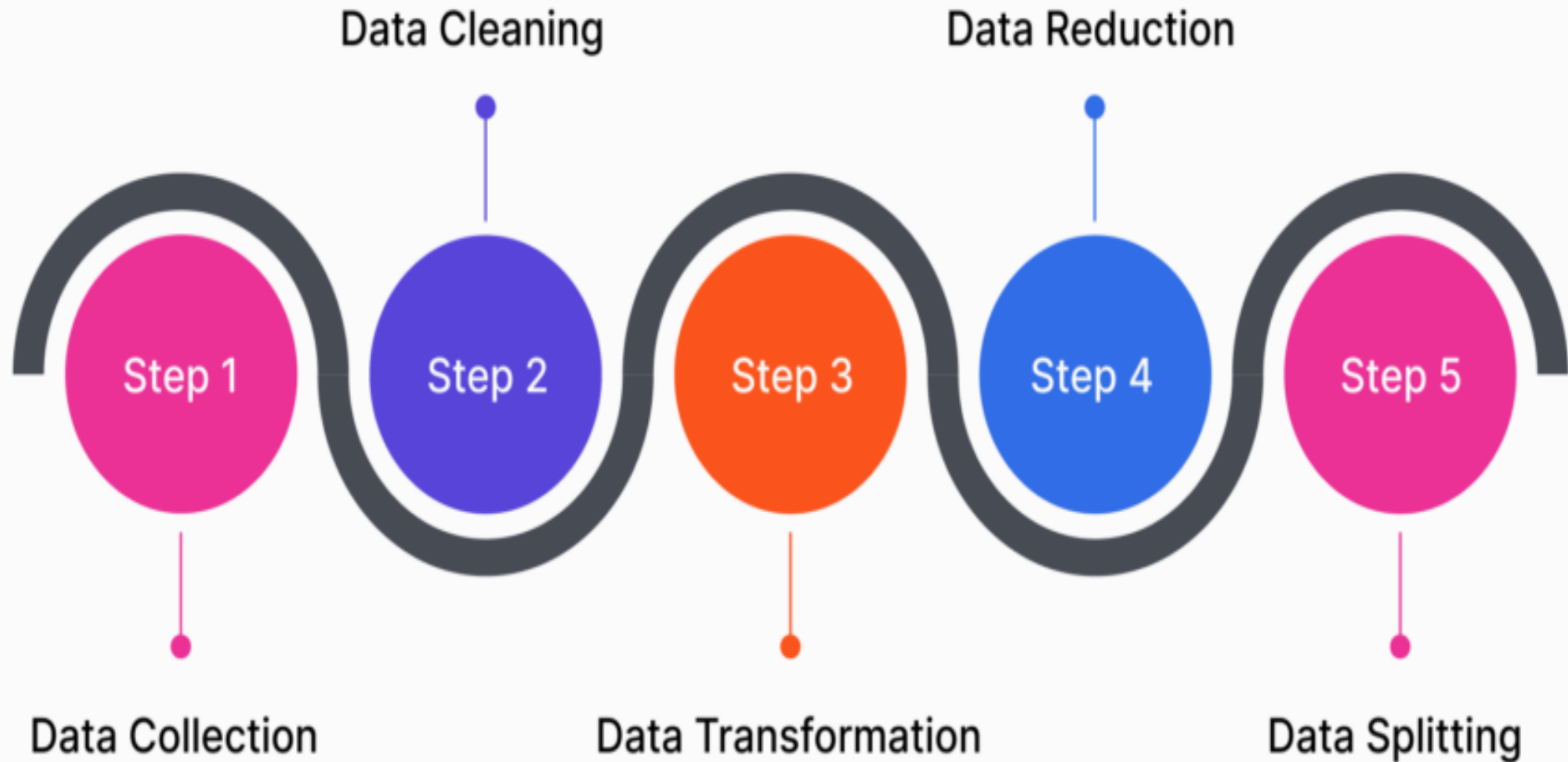
Data Preparation or Pre-processing is the process of cleaning, transforming, and organizing raw data into a structured and usable format before it is analyzed or used in machine learning models.



WHY IS DATA PREPATION IMPORTANT?

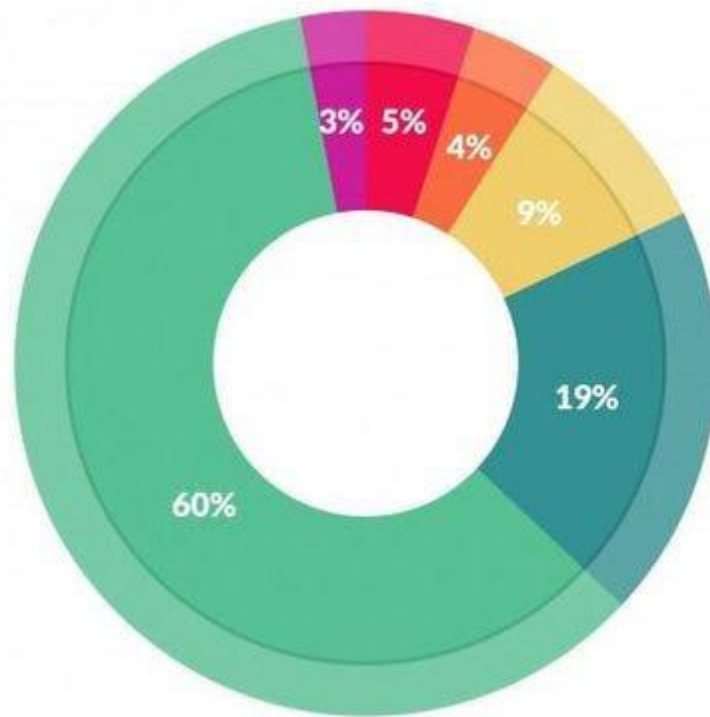
- **Improves Data Quality:** Removes errors, inconsistencies, and duplicates for accurate analysis.
- **Handles Missing Values:** Ensures models are not biased or misled due to incomplete data.
- **Increases Model Accuracy:** Clean and well-prepared data helps machine learning models perform better.
- **Ensures Consistency:** Standardizes data formats for smooth processing and comparison.
- **Enhances Efficiency:** Reduces time and resources spent during analysis or model training.
- **Enables Better Insights:** Organized data helps uncover true patterns and relationships.
- **Supports Feature Engineering:** Makes it easier to derive meaningful features from raw data.
- **Reduces Overfitting:** Eliminates irrelevant or noisy features that can harm model generalization.
- **Ensures Data Compatibility:** Converts data into formats compatible with analytical tools or algorithms.
- **Facilitates Reproducibility:** Well-documented and prepared data supports repeatable and reliable results.

DATA PREPARATION STEPS



DATA PREPARATION

Data scientists spend **80%** of their time on data preparation



What data scientists spend the most time doing

- Building training sets: 3%
- Cleaning and organizing data: 60%
- Collecting data sets; 19%
- Mining data for patterns: 9%
- Refining algorithms: 4%
- Other: 5%

DATA CLEANING

Data cleaning is the process of fixing or removing incorrect, incomplete, duplicate, or inconsistent data from a dataset.

It ensures the data is accurate, consistent, and ready for analysis or machine learning.

Key Tasks in Data Cleaning:

1. Handling Missing Data
2. Outlier Detection and Handling
3. Data Scaling and Transformation
4. Encoding Categorical Variables
5. Handling Duplicates
6. Dealing with inconsistent data

TOOL USED

- **Pandas**
 - Handle missing data
 - Remove duplicates
 - Fix data types
 - Filter and transform columns
- **NumPy**
- **Scikit-learn (sklearn)**
- Preprocessing tools for:
 - Handling missing values (SimpleImputer)
 - Feature scaling (StandardScaler, MinMaxScaler)
 - Encoding categorical variables (LabelEncoder, OneHotEncoder)



DATASET LOADING

```
#Load dataset
```

```
df = pd.read_csv('Titanic.csv')
```

```
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

DATA INSPECTION AND EXPLORATION

Check the data information using **df.info()**

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   PassengerId     891 non-null    int64  
 1   Survived        891 non-null    int64  
 2   Pclass         891 non-null    int64  
 3   Name            891 non-null    object  
 4   Sex             891 non-null    object  
 5   Age             714 non-null    float64 
 6   SibSp           891 non-null    int64  
 7   Parch           891 non-null    int64  
 8   Ticket          891 non-null    object  
 9   Fare            891 non-null    float64 
10   Cabin           204 non-null    object  
11   Embarked        889 non-null    object  
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

DATA INSPECTION AND EXPLORATION

Check the categorical and numerical variables

```
# Categorical columns
cat_col = [col for col in df.columns if df[col].dtype == 'object']
print('Categorical columns :',cat_col)
# Numerical columns
num_col = [col for col in df.columns if df[col].dtype != 'object']
print('Numerical columns :',num_col)

Categorical columns : ['Name', 'Sex', 'Ticket', 'Cabin', 'Embarked']
Numerical columns : ['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
```

DATA INSPECTION AND EXPLORATION

Check total number of unique value in categorical column

```
df[cat_col].nunique()
```

```
Name          891  
Sex            2  
Ticket        681  
Cabin         147  
Embarked       3  
dtype: int64
```

DATA INSPECTION AND EXPLORATION

Check the duplicate rows

```
df.duplicated()
0      False
1      False
2      False
3      False
4      False
...
886     False
887     False
888     False
889     False
890     False
Length: 891, dtype: bool
```

```
df.duplicated().sum()
```

```
0
```

DROP IRRELEVANT COLUMN

```
df_clean = df.drop(columns=['Name', 'Ticket', 'Cabin'])  
df_clean
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	3	male	22.0	1	0	7.2500	S
1	2	1	1	female	38.0	1	0	71.2833	C
2	3	1	3	female	26.0	0	0	7.9250	S
3	4	1	1	female	35.0	1	0	53.1000	S
4	5	0	3	male	35.0	0	0	8.0500	S
...	...	...	...	...	...	...	...	...	...
886	887	0	2	male	27.0	0	0	13.0000	S
887	888	1	1	female	19.0	0	0	30.0000	S
888	889	0	3	female	NaN	1	2	23.4500	S
889	890	1	1	male	26.0	0	0	30.0000	C
890	891	0	3	male	32.0	0	0	7.7500	Q

891 rows × 9 columns

IDENTIFYING MISSING DATA

- A **missing value** is a placeholder in your dataset that represents **incomplete, unavailable, or unknown data** for a particular field.
- **Missing values are usually represented as:**
 - NaN (Not a Number)
 - None (Python null object)
 - Empty strings (‘ ‘)
 - Blank spaces
- Data is missing when values are absent due to **entry errors, system failures, privacy concerns, irrelevance, merging issues, corruption, or non-response**.

TYPES OF MISSING DATA

There are three main types of missing data based on the pattern and cause:

- **MCAR – Missing Completely At Random**
 - The missing data has no relationship with any values observed or unobserved.
 - Data is missing **purely by chance**.
- **MAR – Missing At Random**
 - Missingness is related to observed data, but **not to the missing data itself**.
 - The cause of missing data can be explained by other variables.
- **MNAR – Missing Not At Random**
 - The missing data is related to the missing value itself, not just other variables.
 - There is an **underlying reason** why the value is missing.

METHODS FOR IDENTIFYING MISSING DATA

Function	Description
isnull()	Returns a DataFrame of True for missing values (NaN), else False.
notnull()	Returns True where values are present (non-missing), else False.
isna()	Alias of isnull(); works exactly the same.
info()	Displays DataFrame structure, data types, and non-null counts per column.
isnull().any()	Returns True for columns that have at least one missing value.
isnull().sum()	Counts the number of missing values per column .
isnull().sum().sum()	Returns the total number of missing values in the entire DataFrame.

IDENTIFYING MISSING DATA

Check the missing value column wise for each row using **df.isnull()**

`df.isnull()`

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	False	False	False	False	False	False	False	False	False	False	True	False
1	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	True	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	True	False
...	...	...	...	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False	False	True	False
887	False	False	False	False	False	False	False	False	False	False	False	False
888	False	False	False	False	False	True	False	False	False	False	True	False
889	False	False	False	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False	False	True	False

891 rows × 12 columns

IDENTIFYING MISSING DATA

Check the percentage of total missing value of each column

```
round((df.isnull().sum()/df.shape[0])*100,2)
```

PassengerId	0.00
Survived	0.00
Pclass	0.00
Name	0.00
Sex	0.00
Age	19.87
SibSp	0.00
Parch	0.00
Ticket	0.00
Fare	0.00
Cabin	77.10
Embarked	0.22
dtype: float64	

HANDLING MISSING DATA

Handling missing data means identifying and resolving gaps in a dataset where values are unavailable, to ensure accurate analysis and modeling.

Common methods for handling missing data

Method	Description
Remove Rows/Columns	Drop data entries or fields with missing values
Impute with Mean/Median	Replace missing values with average or median of the column
Impute with Mode	Use most frequent value (for categorical data)
Forward/Backward Fill	Fill missing values using nearby data (good for time series)
Custom Values	Replace with domain-specific placeholders (e.g., 'Unknown')
Model-Based Imputation	Predict missing values using ML algorithms (e.g., regression)

REMOVE MISSING DATA

```
# Remove Rows with Missing Values
```

```
df_dropna_rows = df.dropna(inplace=True)
```

```
df.shape
```

```
(183, 12)
```

```
#Remove Columns with Missing Values
```

```
df_dropna_cols = df.dropna(axis=1, inplace= True)
```

```
df.shape
```

```
(183, 12)
```

IMPUTE WITH MEAN

```
from sklearn.impute import SimpleImputer

#Impute Missing Values with Mean (for numerical columns)
df_mean_impute = df.copy()
mean_imputer = SimpleImputer(strategy='mean')
df_mean_impute['Age'] = mean_imputer.fit_transform(df_mean_impute[['Age']])

df_mean_impute['Age'].isnull().sum()
```

0

IMPUTE WITH MODE

```
from sklearn.impute import SimpleImputer

# Impute Missing Values with Mode (for categorical columns)
df_impute = df.copy()
mode_imputer = SimpleImputer(strategy='most_frequent')
df_impute['Embarked'] = mode_imputer.fit_transform(df_impute[['Embarked']])

df_impute['Embarked'].isnull().sum()

0
```

FORWARD FILL

```
#Forward Fill (uses previous value)  
  
df_ffill = df.copy()  
df_ffill.fillna(method='ffill', inplace=True)  
  
df_ffill.isnull().sum()
```

```
PassengerId      0  
Survived          0  
Pclass           0  
Name             0  
Sex              0  
Age              0  
SibSp            0  
Parch            0  
Ticket           0  
Fare             0  
Cabin            1  
Embarked         0  
dtype: int64
```


BACKWARD FILL

```
#Backward Fill (uses next value)  
df_bfill = df.copy()  
df_bfill.fillna(method='bfill', inplace=True)  
  
df_ffill.isnull().sum()
```

```
PassengerId      0  
Survived         0  
Pclass           0  
Name             0  
Sex              0  
Age              0  
SibSp            0  
Parch            0  
Ticket           0  
Fare             0  
Cabin            1  
Embarked         0  
dtype: int64
```

REPLACE WITH CUSTOM VALUES

```
# Fill with Custom Value (e.g., 'Unknown' for Cabin)  
df_custom = df.copy()  
df_custom['Cabin'].fillna('Unknown', inplace=True)  
  
df_custom['Cabin'].isnull().sum()  
  
0
```

OUTLIER DETECTION

A data point that **varies greatly** from other results is referred to as an outlier. An outlier may also be described as an observation in our data that is **incorrect or abnormal as compared to other observations**.

To find outliers, we can simply plot the **box plot**. Outliers are points that are outside of the minimum and maximum values, as seen in the image below.

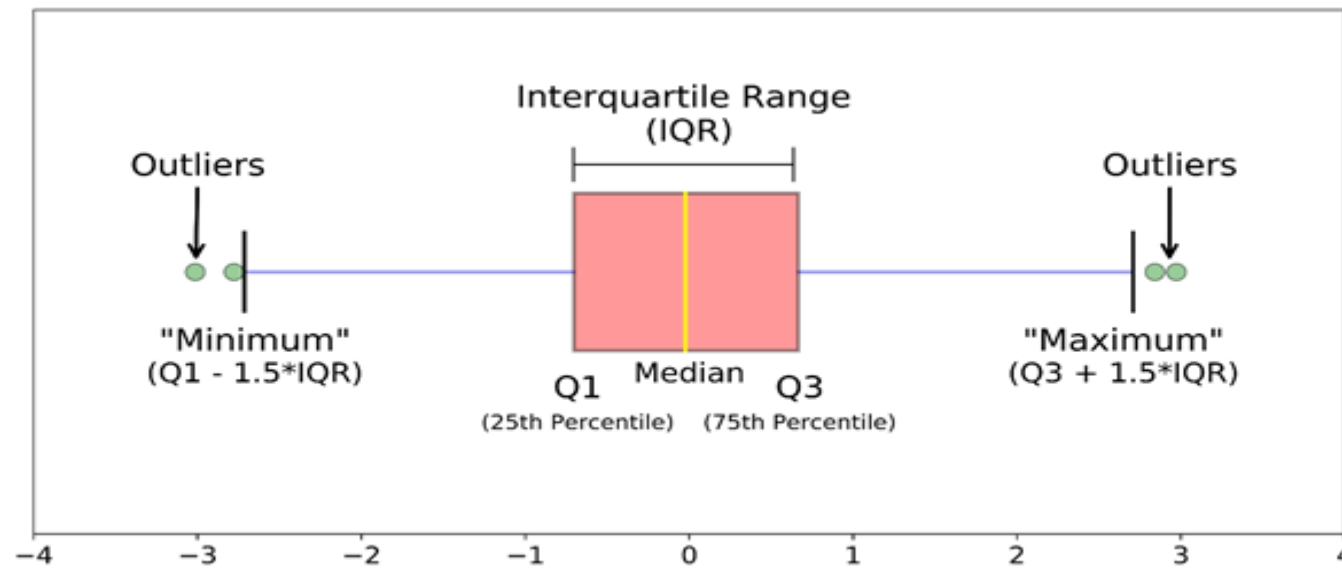


Fig: Box-plot representation

HOW TO MEASURE THE OUTLIER

We can measure the boundary for outliers once we've decided whether outliers are present in the data using the box plot. To measure the outlier, we can use the following methods:

- **If the data is normally distributed**

We can use the empirical formula of Normal Distribution to determine the boundary for outliers if the data is normally distributed.

- Lower Boundary = **Mean — 3* (Standard Deviation)**
- Upper Boundary = **Mean + 3 * (Standard Deviation)**

HOW TO MEASURE THE OUTLIER

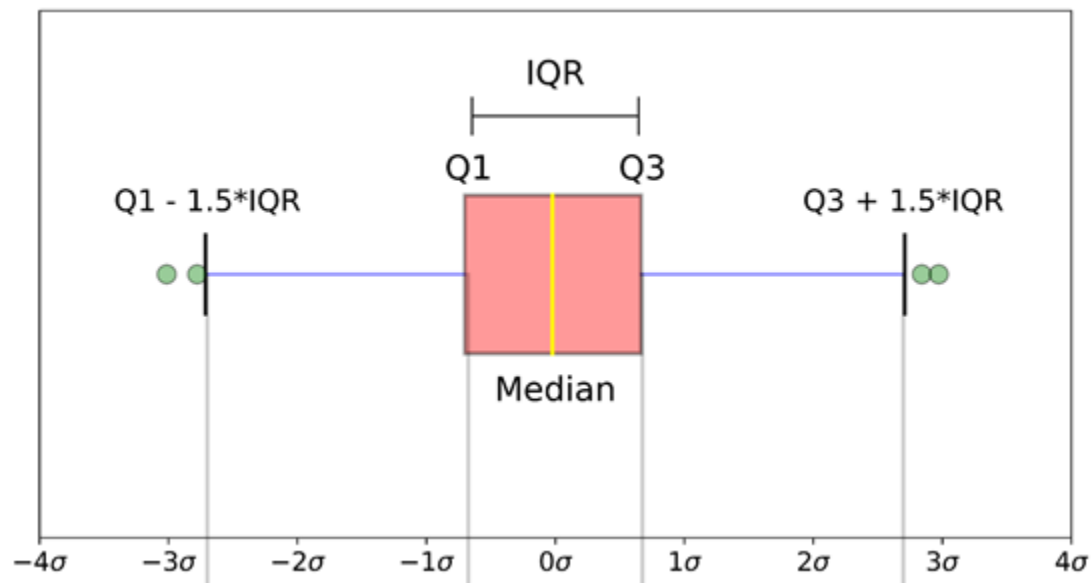


Fig: Box-plot representation of normal distribution

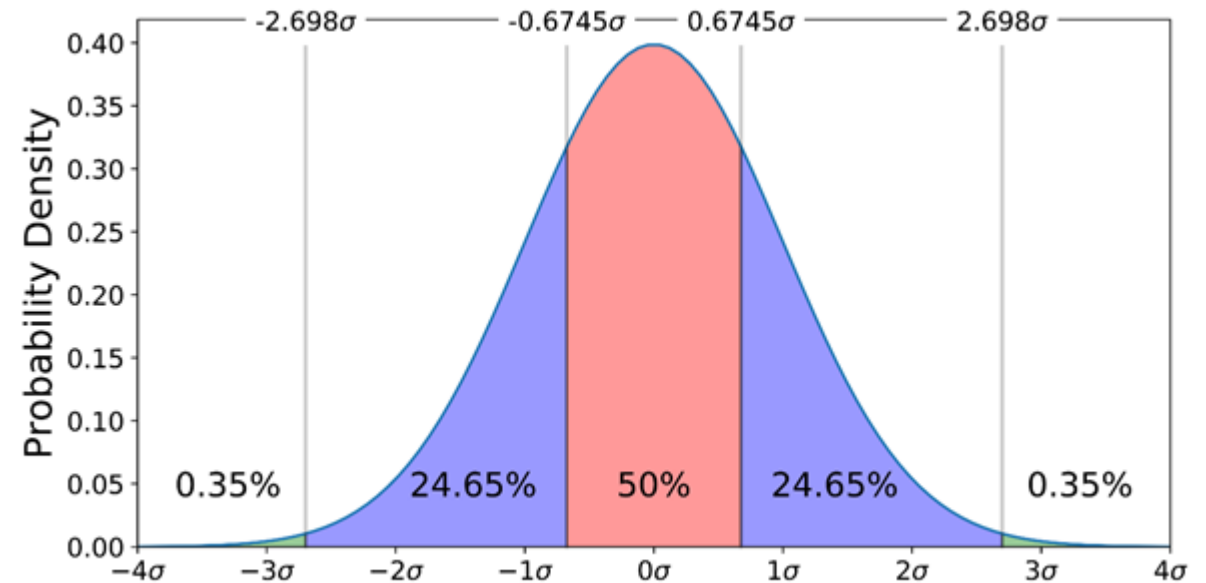
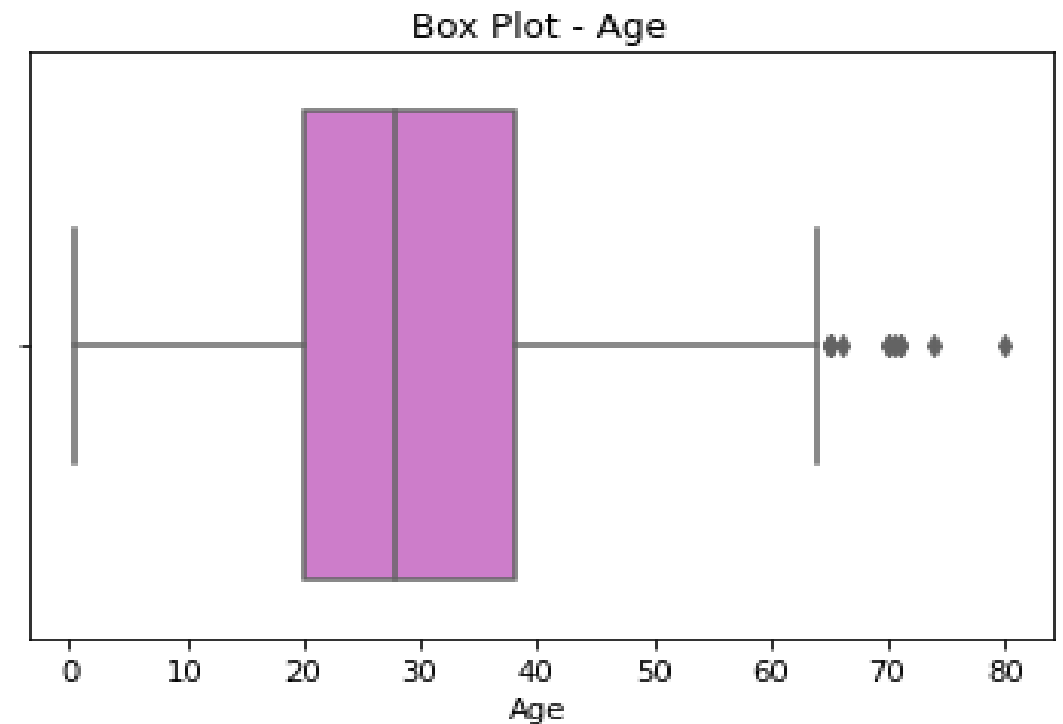


Fig: Gaussian curve representation of normal distribution

HOW TO MEASURE THE OUTLIER

Measure outlier for age column

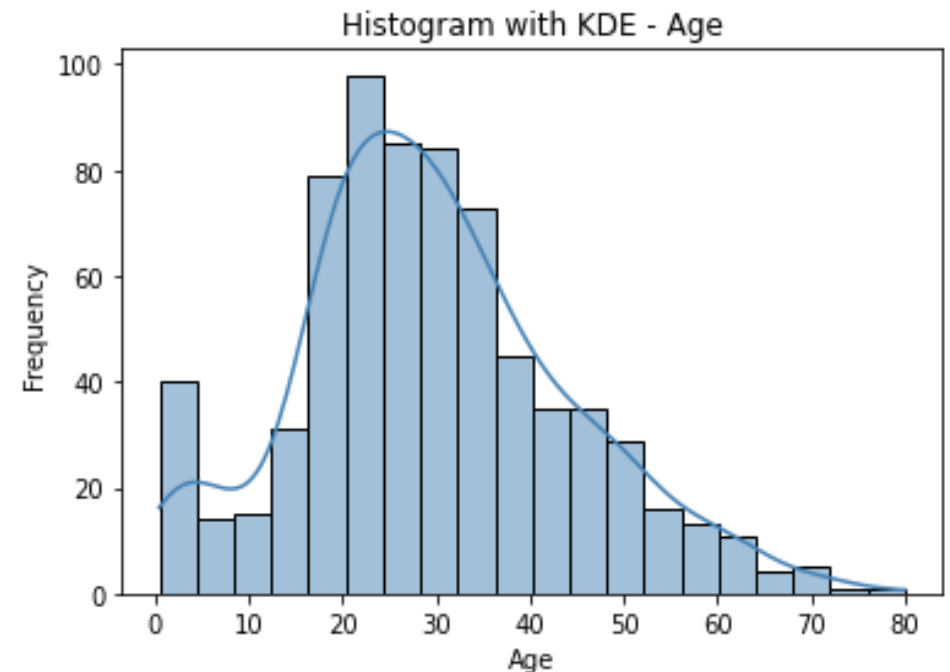
```
plt.figure(figsize=(6, 4))  
sns.boxplot(x=df['Age'], color='orchid')  
plt.title('Box Plot - Age')  
plt.xlabel('Age')  
plt.show()
```



HOW TO MEASURE THE OUTLIER

Measure outlier for age column

```
plt.figure(figsize=(6, 4))  
sns.histplot(df['Age'], kde=True, color='steelblue')  
plt.title('Histogram with KDE - Age')  
plt.xlabel('Age')  
plt.ylabel('Frequency')  
plt.show()
```



HOW TO MEASURE THE OUTLIER

Here the Age columns in normally distributed that means we have to use empirical formula to calculate the Boundaries

```
upper_boundary=df['Age'].mean()+3*df['Age'].std()  
lower_boundary=df['Age'].mean()-3*df['Age'].std()  
print('Lower boundary: ',lower_boundary)  
print('Upper boundary: ',upper_boundary)
```

Lower boundary: -13.88037434994331

Upper boundary: 73.27860964406095

HOW TO MEASURE THE OUTLIER

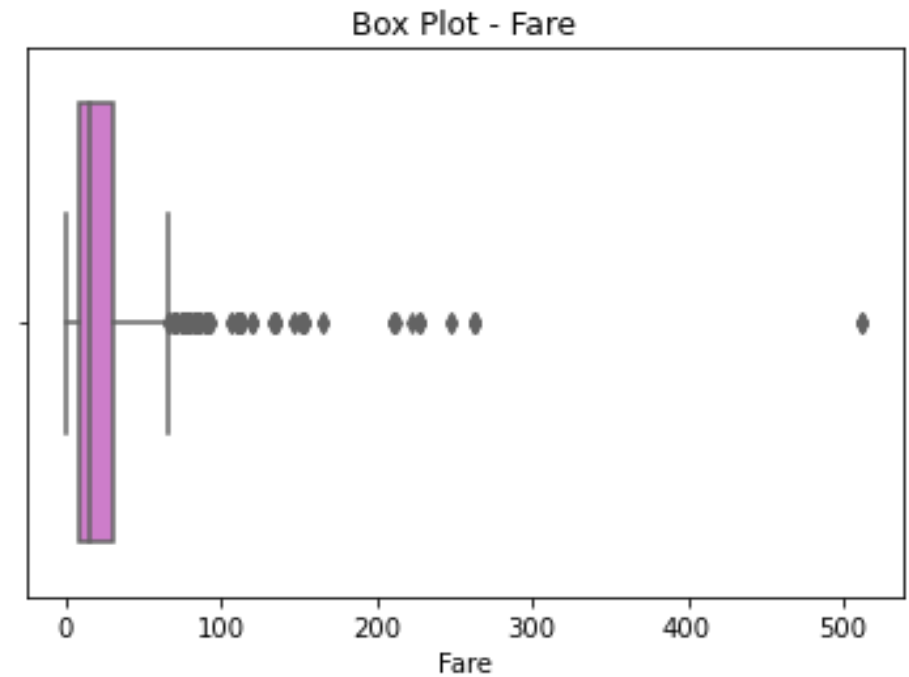
- **If the data is Either left skewed or right skewed**
 - Lower Boundary= First Quartile(Q1/25th percentile) — $(1.5 * IQR)$
 - Upper Boundary = Third Quartile(Q3/75th percentile) $+(1.5 * IQR)$
- **If the outlier's maximum value is extremely high**
 - Lower Boundary= First Quartile(Q1/25th percentile) — $(3 * IQR)$
 - Upper Boundary = Third Quartile(Q3/75th percentile) $+(3 * IQR)$

$$\text{Interquartile Range(IQR)} = Q3(75\text{th percentile}) - Q1(25\text{th percentile})$$

HOW TO MEASURE THE OUTLIER

Measure outlier for fare column column

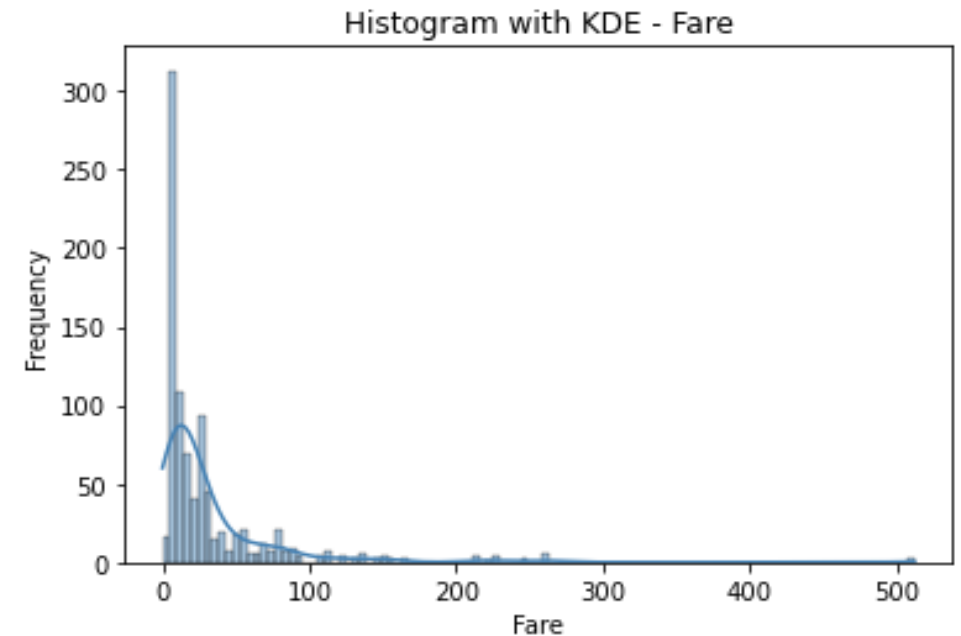
```
plt.figure(figsize=(6, 4))  
sns.boxplot(x=df['Fare'], color='orchid')  
plt.title('Box Plot - Fare')  
plt.xlabel('Fare')  
plt.show()
```



HOW TO MEASURE THE OUTLIER

Measure outlier for fare column

```
plt.figure(figsize=(6, 4))  
sns.histplot(df['Fare'], kde=True, color='steelblue')  
plt.title('Histogram with KDE - Fare')  
plt.xlabel('Fare')  
plt.ylabel('Frequency')  
plt.show()
```



HOW TO MEASURE THE OUTLIER

Here the Data is not Normally distributed and is following the Right Skewed distribution that means we can use the Interquartile Range to measure the boundaries for outliers

```
IQR= df['Fare'].quantile(0.75) - df['Fare'].quantile(0.25)
IQR
```

```
23.0896
```

```
# Calculating the boundaries
lower_fence= df['Fare'].quantile(0.25)-(IQR*1.5)
upper_fence= df['Fare'].quantile(0.75)+(IQR*1.5)
print('Lower boundary: ',lower_fence)
print('Upper boundary: ',upper_fence)
```

```
Lower boundary:  -26.724
```

```
Upper boundary:  65.6344
```

HOW TO MEASURE THE OUTLIER

```
df['Fare'].describe()
```

```
count      891.000000
mean        32.204208
std         49.693429
min          0.000000
25%          7.910400
50%         14.454200
75%         31.000000
max        512.329200
Name: Fare, dtype: float64
```

Here the maximum value of outliers is very high compare to upper boundary that indicates we need to calculate the extreme outlier's boundaries.

HOW TO MEASURE THE OUTLIER

```
#Calculating the extreme boundaries  
lower_bridge= df['Fare'].quantile(0.25)-(IQR*3)  
upper_bridge= df['Fare'].quantile(0.75)+(IQR*3)  
print('Lower boundary: ',lower_bridge)  
print('Upper boundary: ',upper_bridge)
```

```
Lower boundary:  -61.358399999999996  
Upper boundary:  100.2688
```

HANDLING OUTLIER

Following approaches can be used to deal with outliers once we've defined the boundaries for them:

1. Z-Score method
2. IQR method
3. Imputation
4. Remove the observation
5. Model based method

Z – SCORE METHOD

The z-score tells us how many standard deviations a datapoint is from the mean of the dataset. It is a way to understand whether a value is typical or unusual compared to the rest of the data.

$$Z = \frac{x - \mu}{\sigma}$$

Diagram illustrating the Z-score formula with annotations:

- Value**: Points to x (the numerator).
- Mean**: Points to μ (the denominator).
- SD** (Standard Deviation): Points to σ (the denominator).

Z – SCORE METHOD

```
from scipy.stats import zscore

# Drop missing Fare values
df = df[['Fare']].dropna()

# Calculate Z-score
df['Fare_z'] = zscore(df['Fare'])

# Filter out outliers (Z-score beyond ±3)
df_filtered = df[(df['Fare_z'] >= -3) & (df['Fare_z'] <= 3)]

df_filtered
```

	Fare	Fare_z
0	7.2500	-0.502445
1	71.2833	0.786845
2	7.9250	-0.488854
3	53.1000	0.420730
4	8.0500	-0.486337
...	...	...
886	13.0000	-0.386671
887	30.0000	-0.044381
888	23.4500	-0.176263
889	30.0000	-0.044381
890	7.7500	-0.492378

871 rows × 2 columns

Z – SCORE METHOD

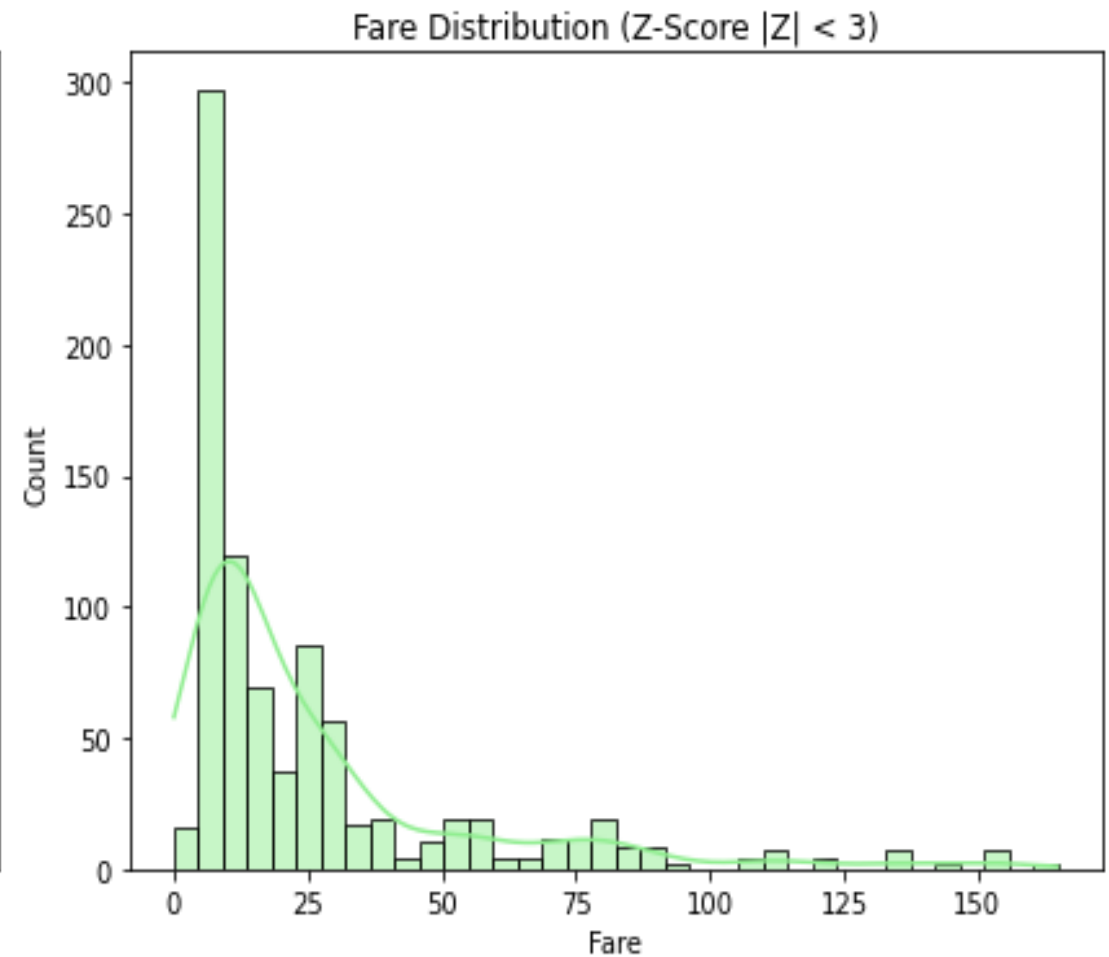
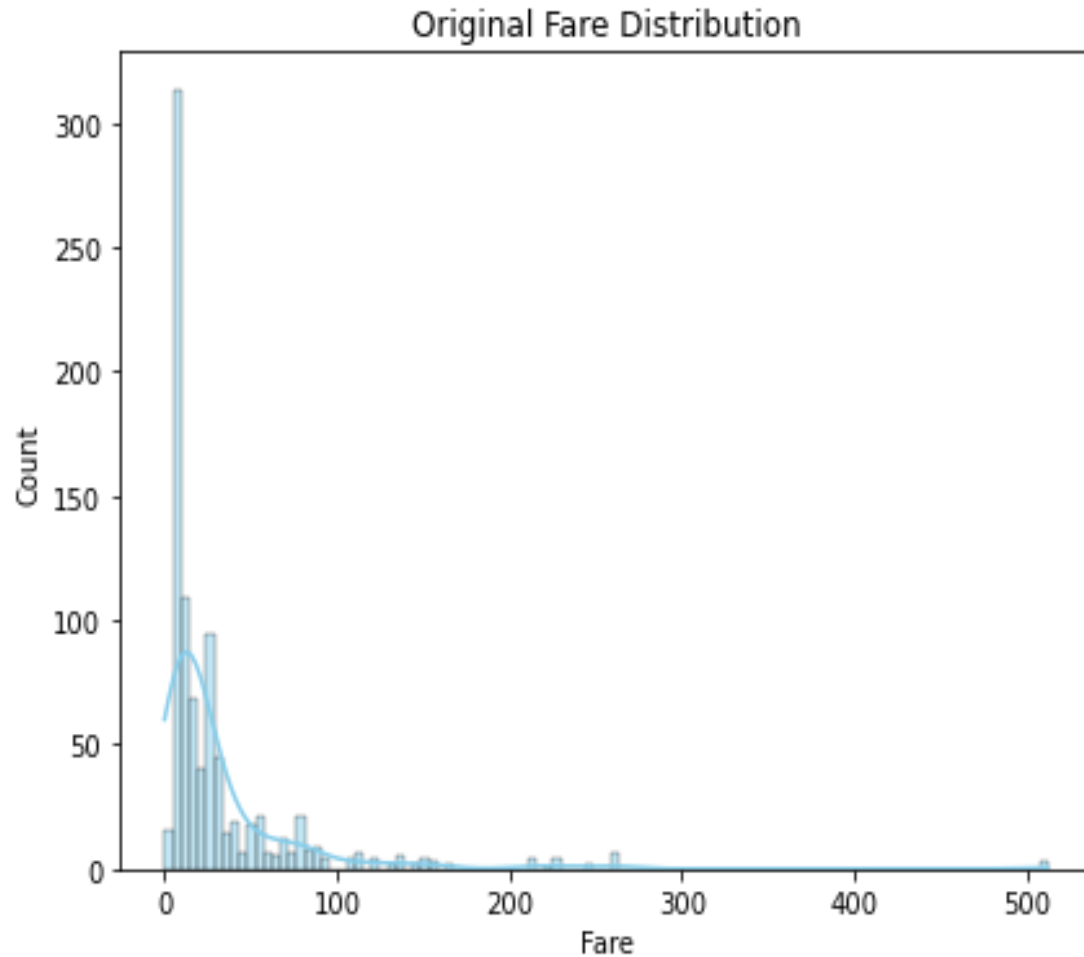
```
# Plotting
plt.figure(figsize=(12, 5))

# Before removing outliers
plt.subplot(1, 2, 1)
sns.histplot(df['Fare'], kde=True, color='skyblue')
plt.title('Original Fare Distribution')

# After removing outliers
plt.subplot(1, 2, 2)
sns.histplot(df_filtered['Fare'], kde=True, color='lightgreen')
plt.title('Fare Distribution (Z-Score  $|Z| < 3$ )')

plt.tight_layout()
plt.show()
```

Z – SCORE METHOD



IQR METHOD

```
# Calculate Q1, Q3, and IQR
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

# Calculate bounds
lower_bound = Q1 - 3 * IQR
upper_bound = Q3 + 3 * IQR

# Filter out outliers
df_filtered = df[(df['Fare'] >= lower_bound) & (df['Fare'] <= upper_bound)]
```

IQR METHOD

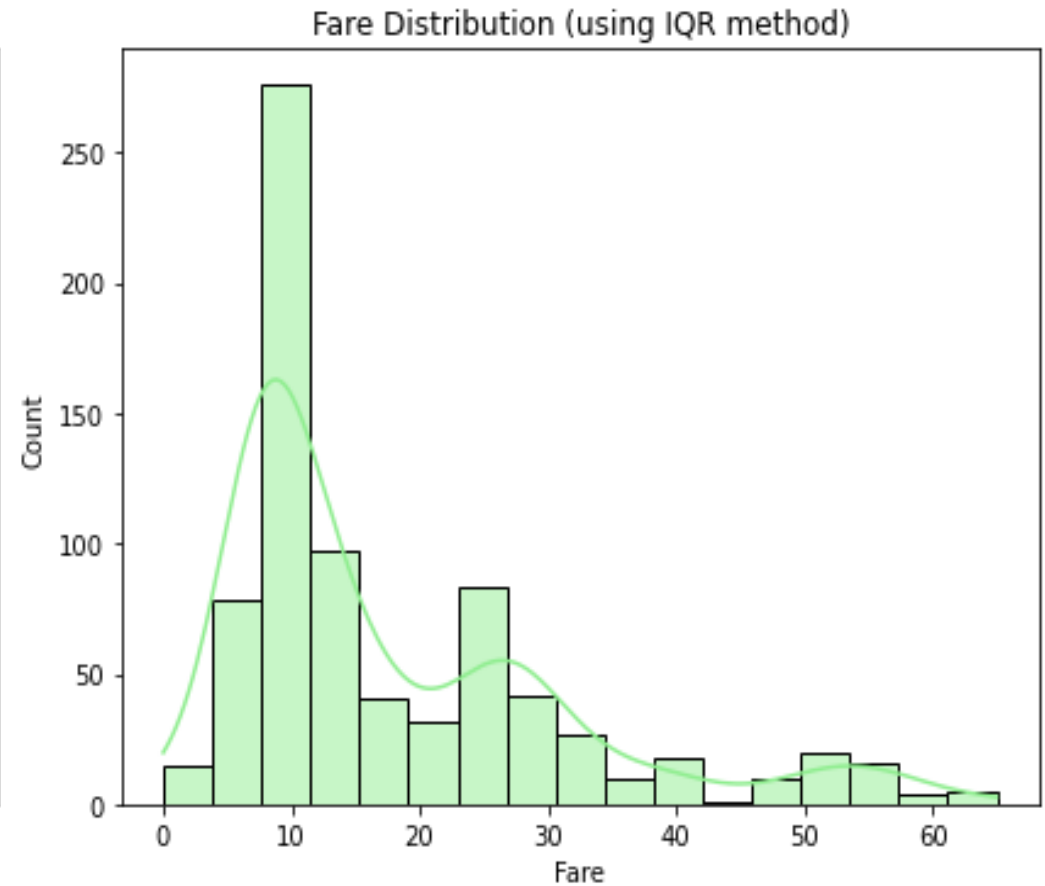
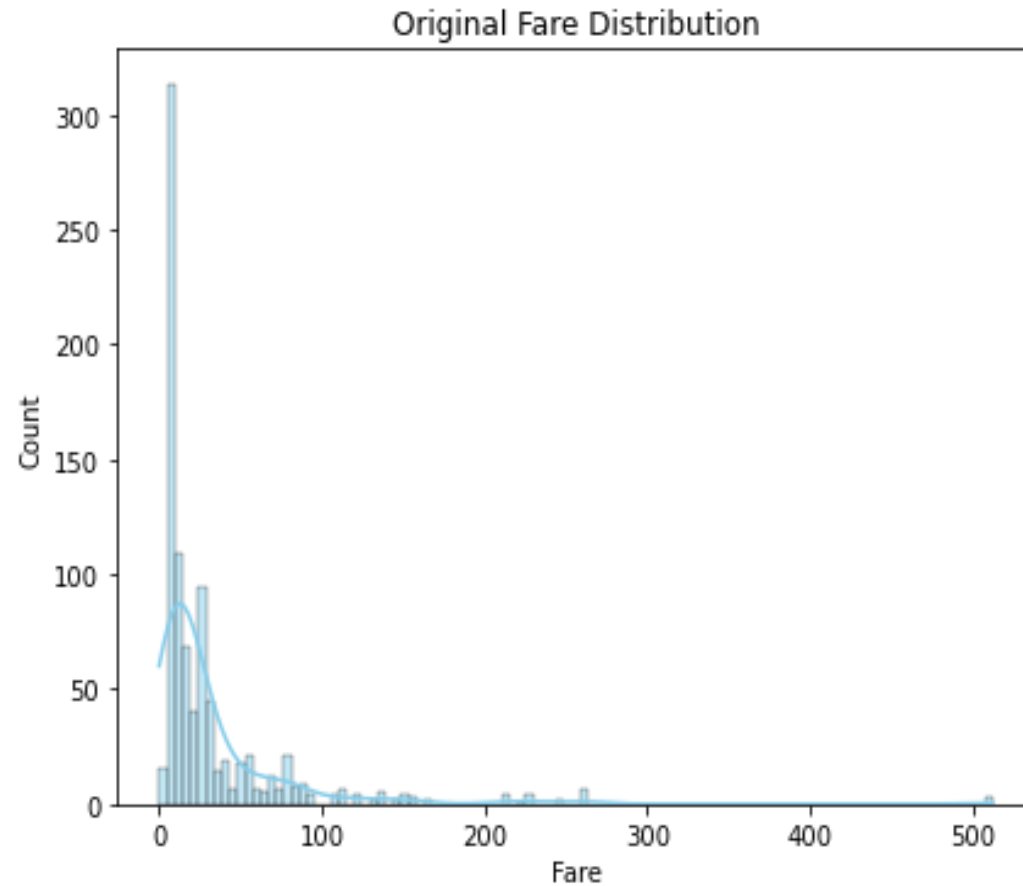
```
# Plotting
plt.figure(figsize=(12, 5))

# Before removing outliers
plt.subplot(1, 2, 1)
sns.histplot(df['Fare'], kde=True, color='skyblue')
plt.title('Original Fare Distribution')

# After removing outliers
plt.subplot(1, 2, 2)
sns.histplot(df_iqr_filtered['Fare'], kde=True, color='lightgreen')
plt.title('Fare Distribution (using IQR method)')

plt.tight_layout()
plt.show()
```

IQR METHOD



IMPUTAION METHOD

```
median_fare = df_filtered['Fare'].median()  
df['Fare_replaced'] = df['Fare'].apply(  
    lambda x: median_fare if x > upper_bound or x < lower_bound else x)
```

IMPUTAION METHOD

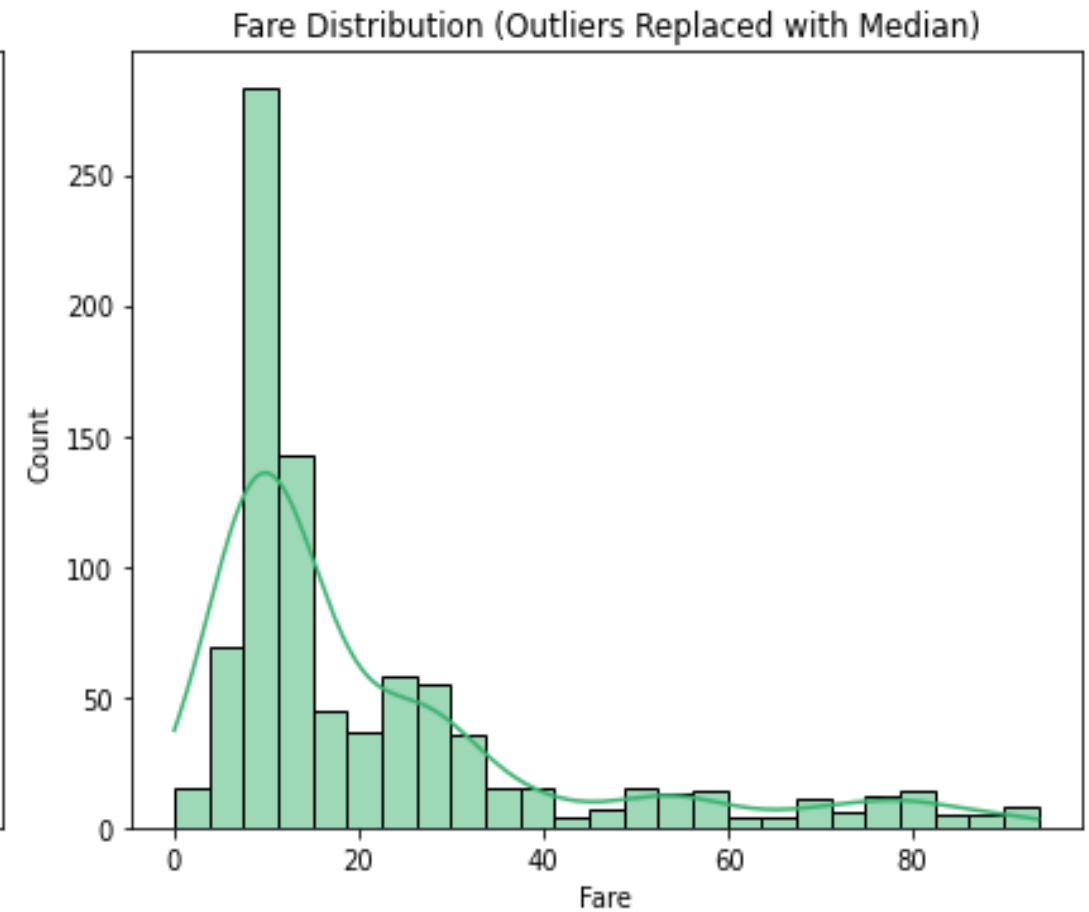
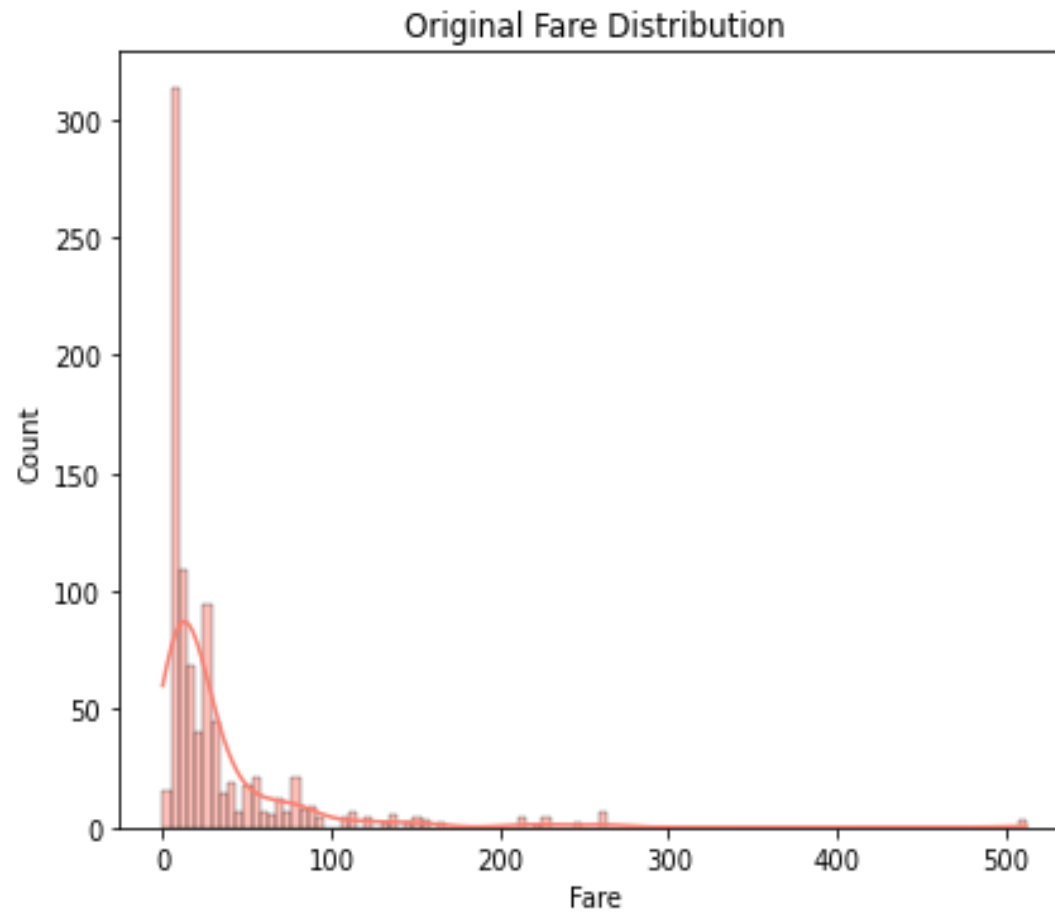
```
# Plotting
plt.figure(figsize=(12, 5))

# Original Fare Distribution
plt.subplot(1, 2, 1)
sns.histplot(df['Fare'], kde=True, color='salmon')
plt.title('Original Fare Distribution')
plt.xlabel('Fare')

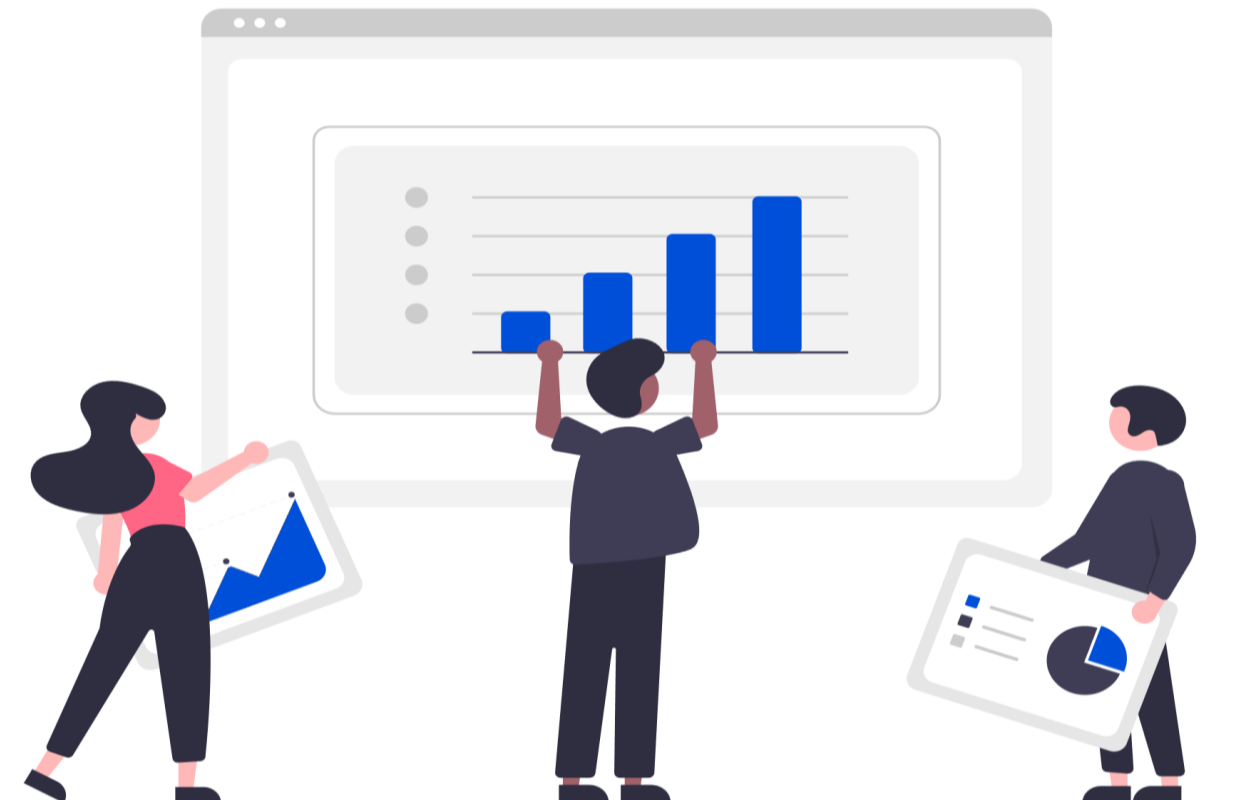
# Fare Distribution After Replacement
plt.subplot(1, 2, 2)
sns.histplot(df['Fare_replaced'], kde=True, color='mediumseagreen')
plt.title('Fare Distribution (Outliers Replaced with Median)')
plt.xlabel('Fare')

plt.tight_layout()
plt.show()
```


IMPUTAION METHOD



FEATURE SCALING



FEATURE SCALING

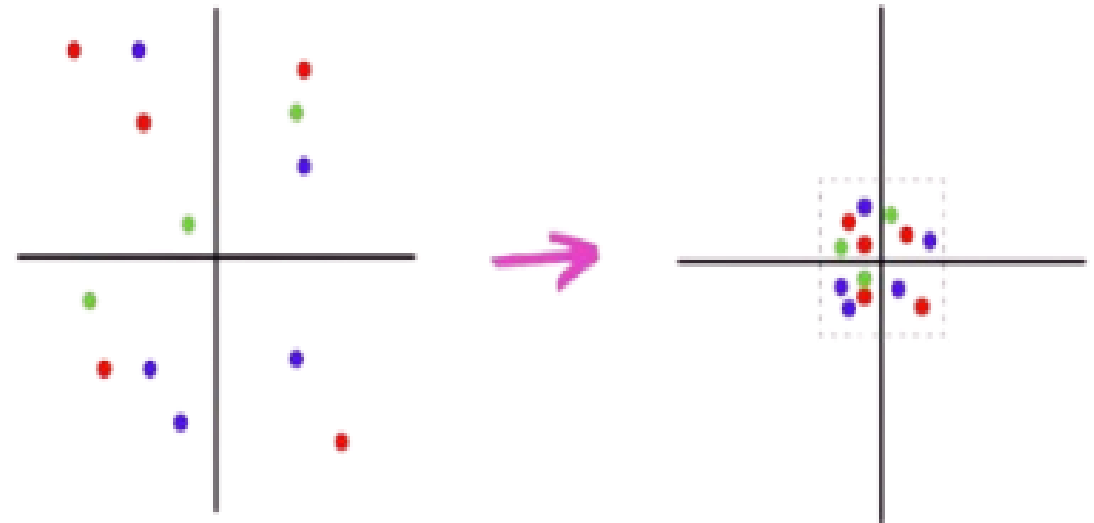
Feature scaling is a vital pre-processing step in machine learning that involves transforming **numerical features to a common scale.**

Role of Feature Scaling in Machine Learning:

- Makes features comparable.
- Enhances accuracy and model performance.
- Prevents feature dominance due to scale.
- Essential for distance-based algorithms.
- Improves interpretability in some models.
- Standardizes data range for better training.
- Reduces computational complexity in some cases

FEATURE SCALING TECHNIQUES

1. Standardization (Z – score scaling)
2. Normalization (Min – Max scaling)
3. Robust Scaling
4. Binarization



STANDARIZATION

Standardization (also called Z-score normalization) is a feature scaling technique that transforms data to have:

- **Mean = 0**
- **Standard Deviation = 1**

Standardised Value

Original Value

Sample Mean

Sample Standard Deviation

$$x' = \frac{x - \mu}{\sigma}$$

Fig : standardization formula

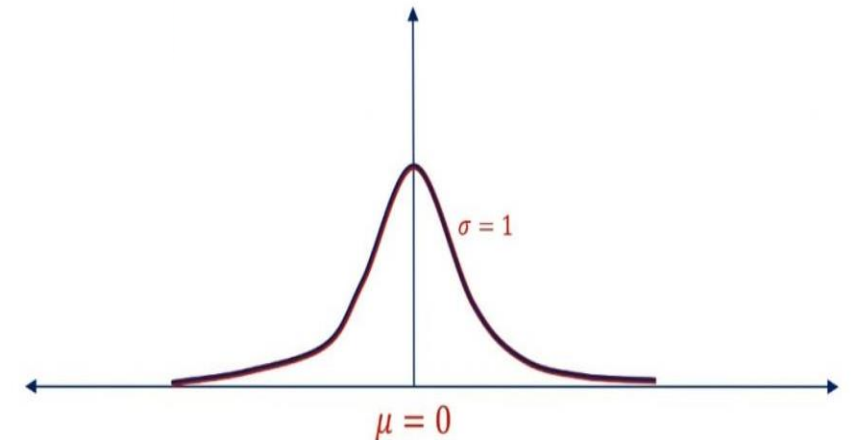


Fig : standard normal distribution

STANDARIZATION

```
from sklearn.preprocessing import StandardScaler

# Initialize StandardScaler
scaler = StandardScaler()

# Fit and transform
scaled_data = scaler.fit_transform(age_clean)

# Convert back to DataFrame
scaled_df = pd.DataFrame(scaled_data, columns=age_clean.columns)

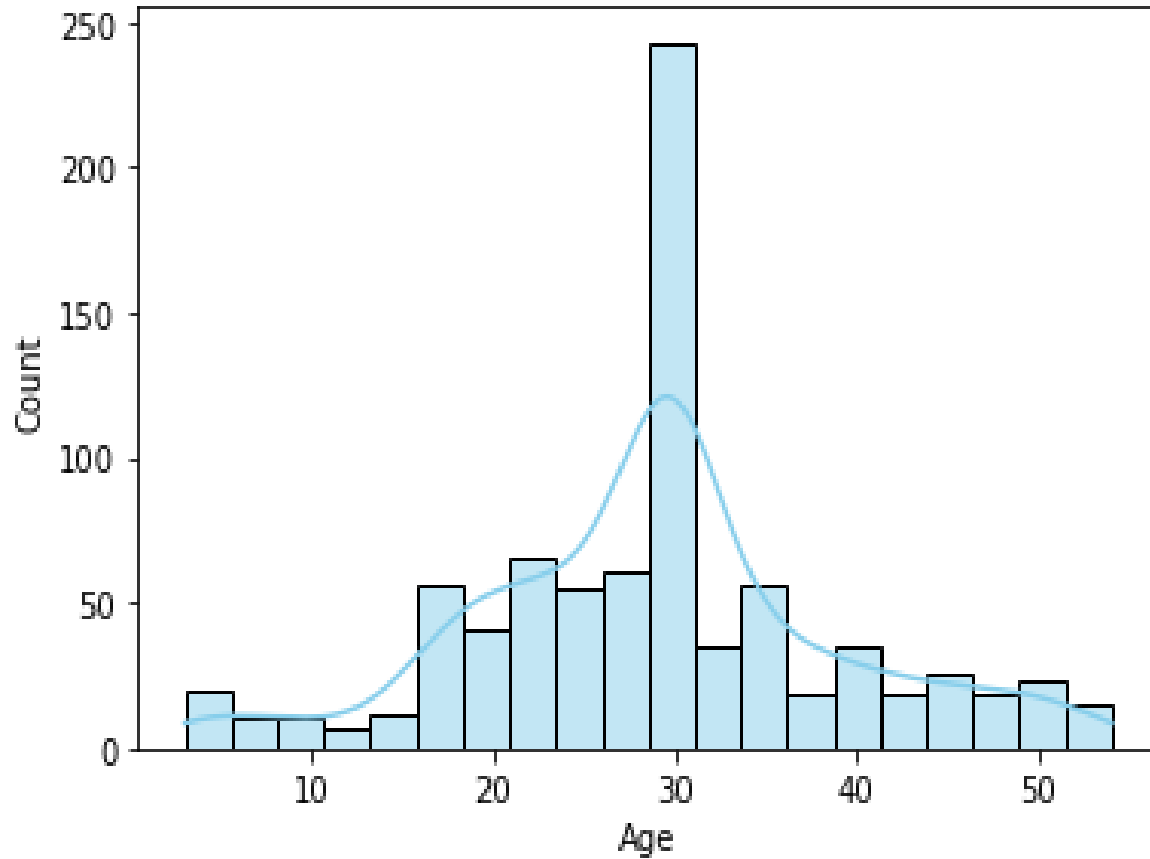
scaled_df.sample(10)
```

STANDARIZATION

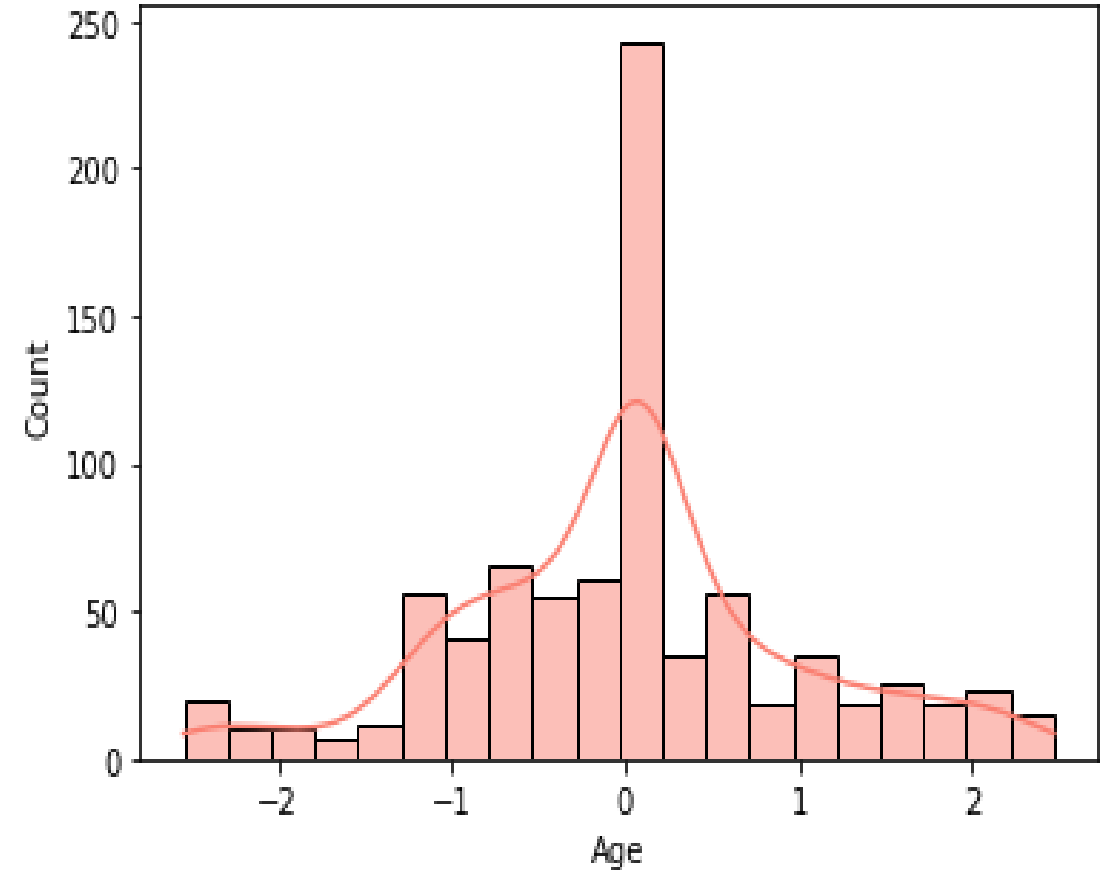
```
# Plot a single feature before and after scaling with KDE  
sns.histplot(age_clean['Age'], kde=True, color='skyblue')  
plt.title("Age - Before Scaling")  
plt.show()  
  
sns.histplot(scaled_df['Age'], kde=True, color='salmon')  
plt.title("Age - After Scaling")  
plt.show()
```

STANDARIZATION

Age - Before Scaling

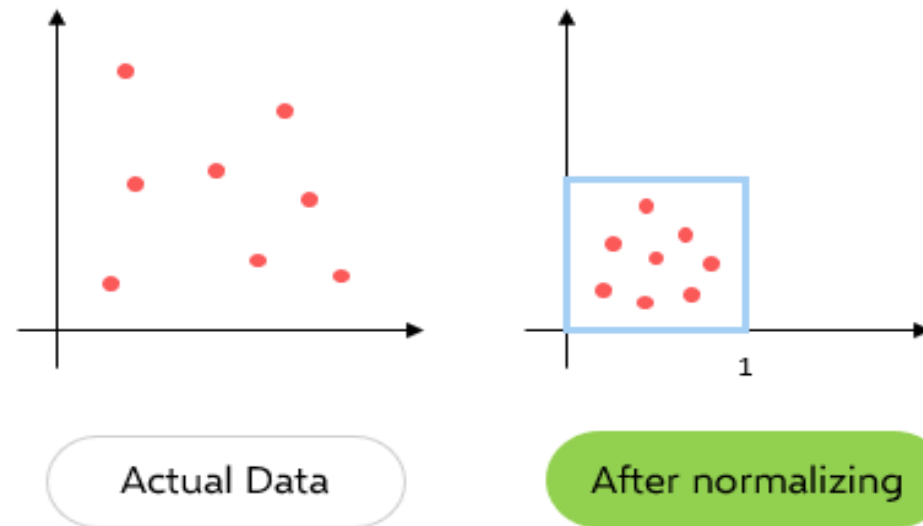


Age - After Scaling



NORMALIZATION

- Normalization, is a data preprocessing technique employed to standardize the values of features in a dataset, bringing them to a common scale.
- In this technique values are shifted and rescaled so that they end up ranging between 0 and 1.
- It is also known as Min-Max scaling.



NORMALIZATION

Here is the formula for normalization :

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Here, **X_{max}** and **X_{min}** are the maximum and the minimum values of the feature, respectively.

- When the value of X is **the minimum value** in the column, the numerator will be 0, and hence **X' is 0**.
- On the other hand, when the value of X is **the maximum value** in the column, the numerator is equal to the denominator, and thus the value of **X' is 1**.
- If the value of X is **between the minimum and the maximum value**, then the value of **X' is between 0 and 1**

NORMALIZATION

```
from sklearn.preprocessing import MinMaxScaler

# Apply Min-Max Normalization
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(num_cols)

# Convert back to DataFrame
normalized_df = pd.DataFrame(normalized_data, columns=num_cols.columns)

# View results
normalized_df.head()
```

NORMALIZATION

```
from sklearn.preprocessing import MinMaxScaler

# Apply Min-Max Normalization
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(age_clean)

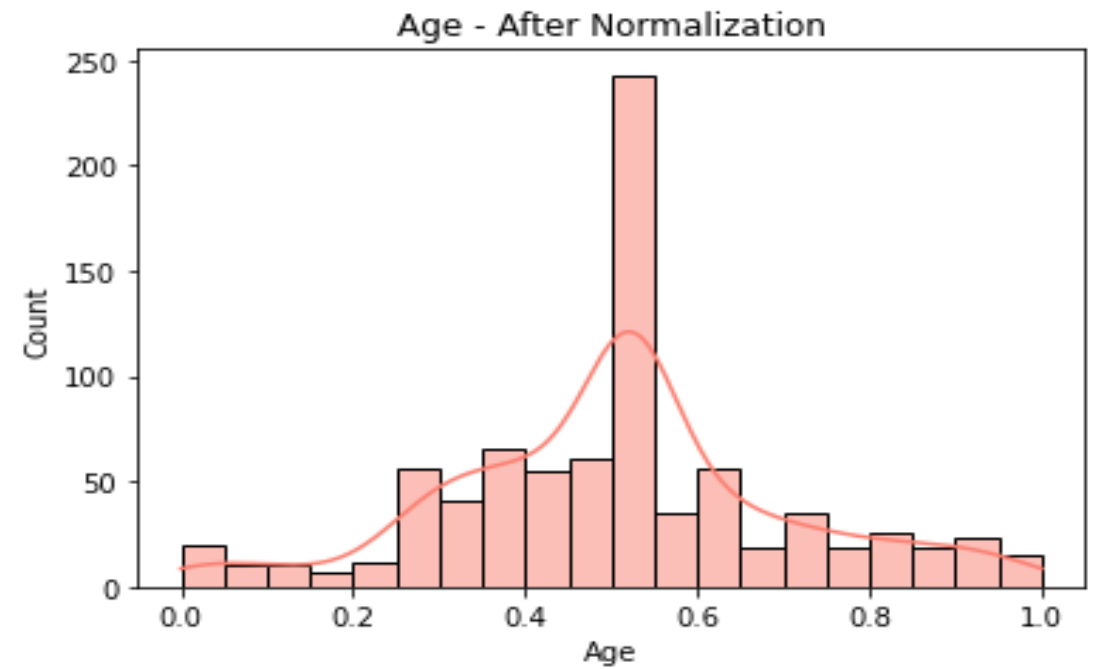
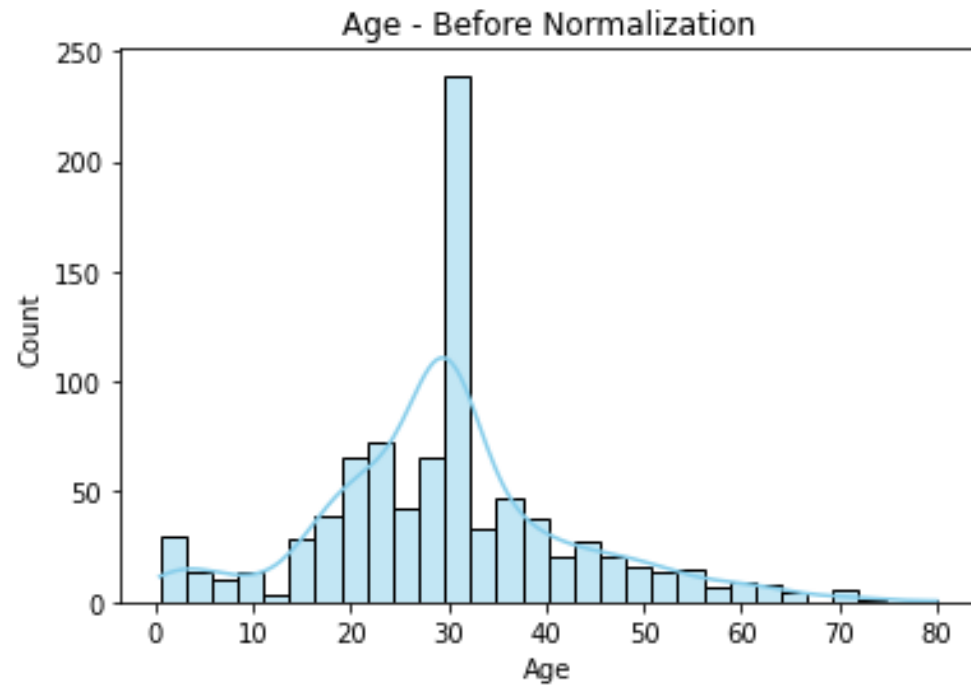
# Convert back to DataFrame
normalized_df = pd.DataFrame(normalized_data, columns=age_clean.columns)

# View results
normalized_df.head()
```

NORMALIZATION

```
# Plot a single feature before and after scaling with KDE  
sns.histplot(num_cols['Age'], kde=True, color='skyblue')  
plt.title("Age - Before Scaling")  
plt.show()  
  
sns.histplot(normalized_df['Age'], kde=True, color='salmon')  
plt.title("Age - After Scaling")  
plt.show()
```

NORMALIZATION



BINARIZATION

Binarization is a data preprocessing technique that **converts numerical values into binary values** (0 or 1) based on a **threshold**.

For each value x :

$$x_{\text{binary}} = \begin{cases} 1 & \text{if } x > \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

BINARIZATION

```
from sklearn.preprocessing import Binarizer

# Drop missing age values
df_age = df[['Age']].dropna().copy()

# Binarize age: 1 if age > 18 (adult), else 0 (child)
binarizer = Binarizer(threshold=18)
binary_age = binarizer.fit_transform(df_age)

# Add binary column to DataFrame
df_age['age_binary'] = binary_age

df_age.sample(5)
```

Output

	Age	age_binary
544	50.0	1.0
404	20.0	1.0
651	18.0	0.0
883	28.0	1.0
856	45.0	1.0

ROBUST SCALING

Robust Scaling reduces the impact of outliers by scaling data using **median and interquartile range (IQR)**, which makes it fit to extreme values.

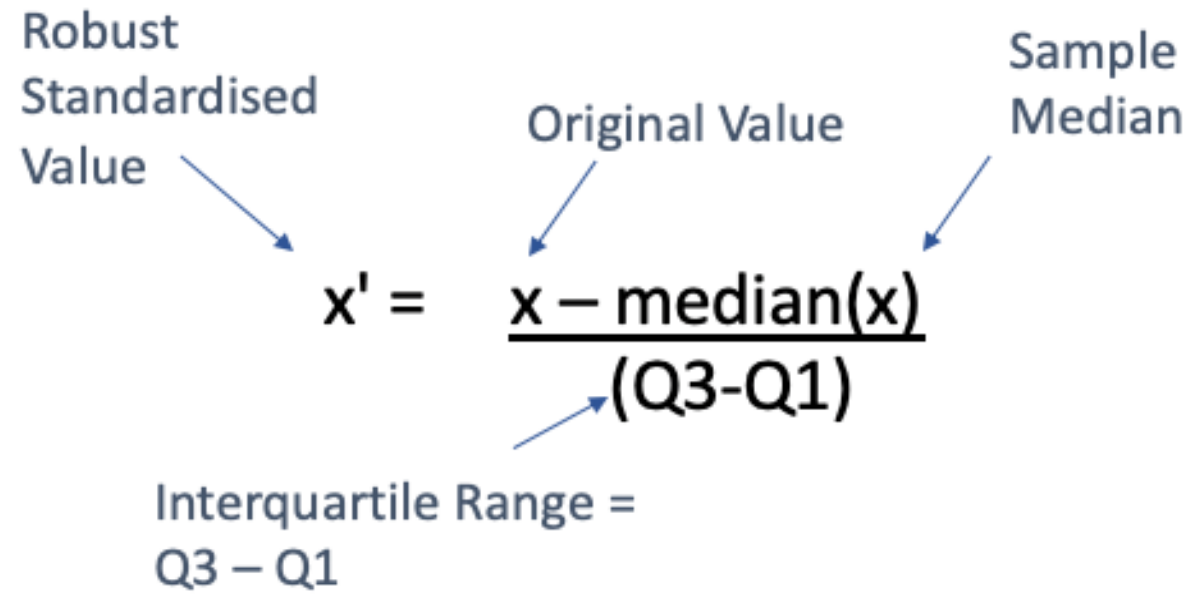
Robust Standardised Value

Original Value

Sample Median

$$x' = \frac{x - \text{median}(x)}{(Q3 - Q1)}$$

Interquartile Range = $Q3 - Q1$



ROBUST SCALING

```
from sklearn.preprocessing import RobustScaler

# Apply Robust Scaling
scaler = RobustScaler()
scaled_data = scaler.fit_transform(num_cols)

# Convert to DataFrame
scaled_df = pd.DataFrame(scaled_data, columns=num_cols.columns)

# Show before vs after for 'fare'
print("Before scaling - Fare stats:")
print(num_cols['Fare'].describe())

print("\nAfter Robust scaling - Fare stats:")
print(scaled_df['Fare'].describe())
```

ROBUST SCALING

Output

Before scaling - Fare stats:

count	891.000000
mean	32.204208
std	49.693429
min	0.000000
25%	7.910400
50%	14.454200
75%	31.000000
max	512.329200

Name: Fare, dtype: float64

After Robust scaling - Fare stats:

count	891.000000
mean	0.768745
std	2.152200
min	-0.626005
25%	-0.283409
50%	0.000000
75%	0.716591
max	21.562738

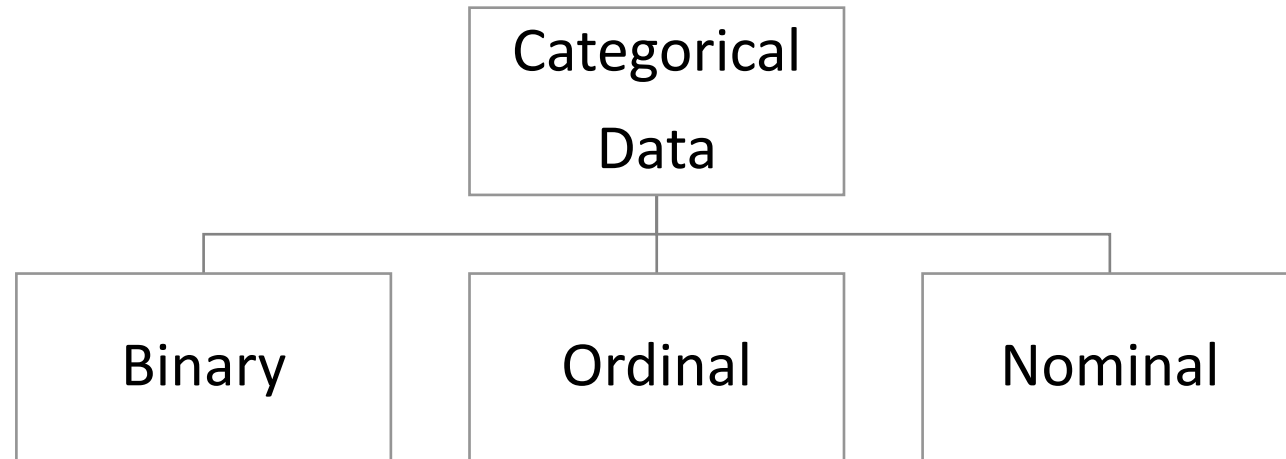
Name: Fare, dtype: float64

SUMMARY OF SCALING TECHNIQUES

Method	Output Range	Sensitive to Outliers	When to Use
Standardization	No fixed range	✓ Yes	- When features are normally distributed- For linear models, PCA
Normalization	0, 1	✓ Yes	- For distance-based models (KNN, SVM)- Neural networks
Robust Scaling	No fixed range	✗ No	- When data contains outliers- Works well with real-world noisy data
Binarization	0 or 1	✗ No	- When binary features are needed- For threshold-based feature logic

ENCODING

Encoding in data preprocessing means converting categorical (textual) data into numerical values



Example :

- Yes / No
- True / False

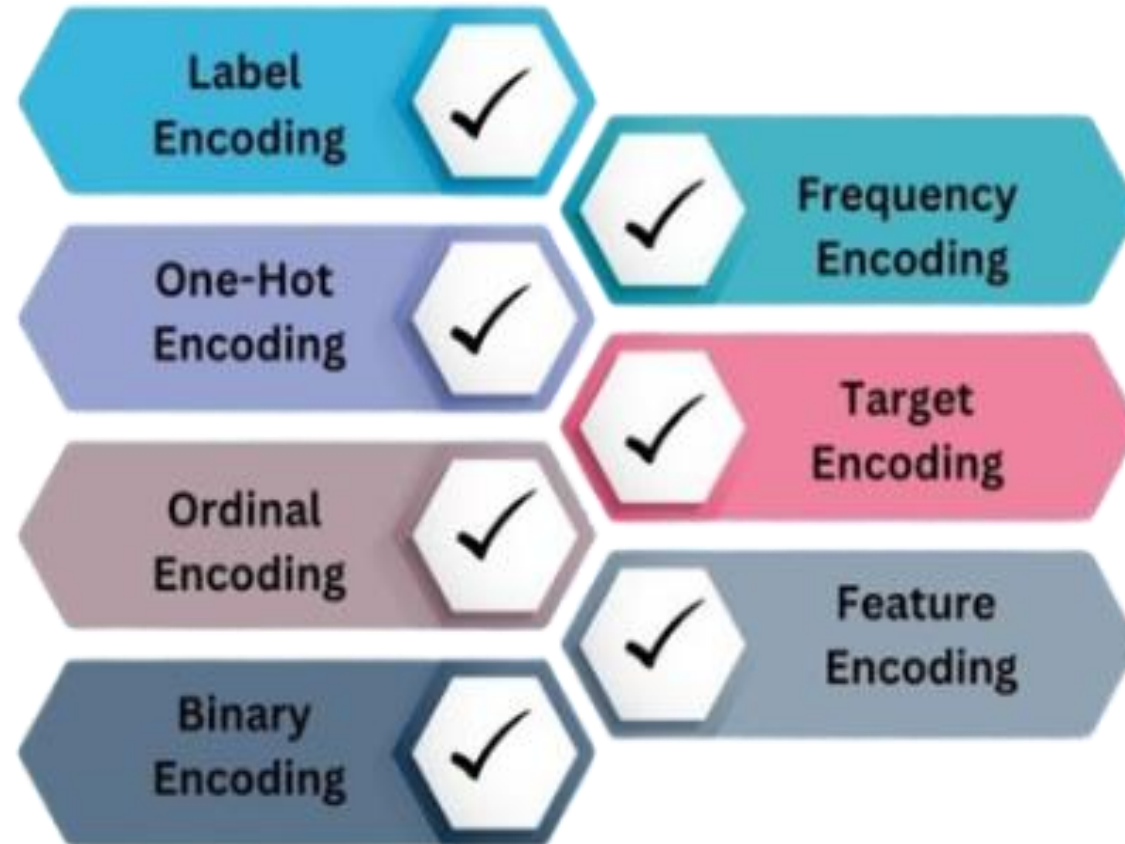
Specified groups
like low, medium,
high

Unordered groups
like cat, dog, tiger

WHY WE NEED ENCODING ?

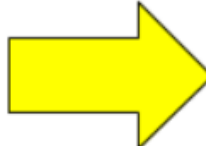
- **Machine learning models require numerical input.**
- **Encoding converts categories into numbers**, making them usable by algorithms.
- It enables the model to **interpret and process features mathematically**.
- **Categorical features like gender, city, or product type** need encoding to be useful.
- It helps avoid errors caused by incompatible data types during training.
- Encoding improves **model accuracy and performance** by making categories learnable.
- It allows models to **recognize relationships or differences** between categories.
- Without encoding, models may **misinterpret or ignore categorical information**.

TYPES OF ENCODING



ONE-HOT ENCODING

One-hot encoding is a technique, which converts each feature category into a **binary (0 or 1)** feature. This approach **creates binary columns for each category in a categorical feature**. That category is present if it is 1 and else 0.



Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

Fig : example of one-hot encoding

ONE-HOT ENCODING

Advantages:

- Treats **all categories equally** without implying any order.
- Easy to apply using libraries like pandas or scikit-learn.
- Each binary column clearly represents a single category.

Disadvantages:

- Can create many columns for features with many categories.
- Most values become 0, leading to **inefficient memory usage**.
- More features can **increase model complexity and overfitting**.
- **Not Suitable for Ordinal Data**

ONE-HOT ENCODING

Example

```
from sklearn.preprocessing import OneHotEncoder

# Display initial categorical columns
cat_cols=df.select_dtypes(include='object')

# Make a copy of the original 'Sex' column
df_copy = cat_cols[['Sex']].copy()

# Initialize and fit OneHotEncoder
enc = OneHotEncoder(sparse=False)
encoded_array = enc.fit_transform(df_copy)
# # Get the encoded category names
encoded_feature_names = enc.get_feature_names_out(['Sex'])

# # Convert encoded array to DataFrame with column names
encoded_df = pd.DataFrame(encoded_array, columns=encoded_feature_names)

# # Concatenate original and encoded columns
df_final = pd.concat([df_copy, encoded_df], axis=1)
# # Show the result
df_final.head()
```

ONE-HOT ENCODING

	Sex	Sex_female	Sex_male
0	male	0.0	1.0
1	female	1.0	0.0
2	female	1.0	0.0
3	female	1.0	0.0
4	male	0.0	1.0

Add two new column in
categorical data

ORDINAL ENCODING

Ordinal encoding converts categorical values into numeric form by assigning a **unique integer** to each category. It's best suited when the categorical variable has a **natural order or ranking** (e.g., small < medium < large).

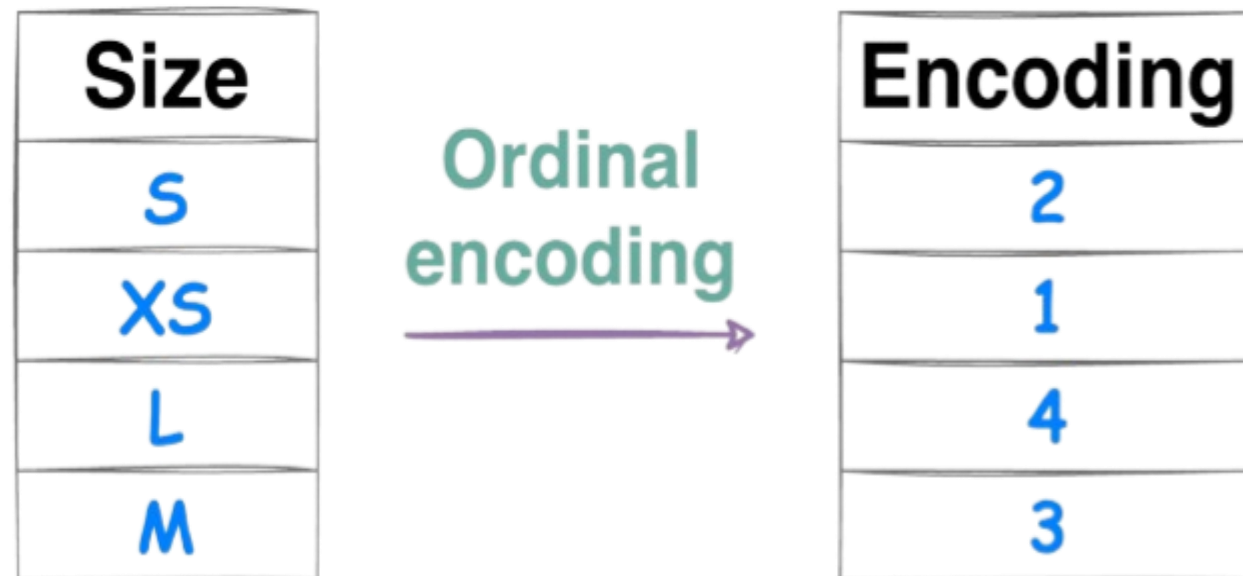


Fig : example of ordinal encoding

ORDINAL ENCODING

```
from sklearn.preprocessing import OrdinalEncoder

encoder = OrdinalEncoder(categories=[['female', 'male']])
df['Sex_encoded'] = encoder.fit_transform(df[['Sex']])
print("\nEncoded Data:")
print(df[['Sex', 'Sex_encoded']].head())
```

Encoded Data:

	Sex	Sex_encoded
0	male	1.0
1	female	0.0
2	female	0.0
3	female	0.0
4	male	1.0

LABEL ENCODING

- **Label Encoding** is a technique that converts **categorical (text) data into numbers** by assigning a **unique integer** to each unique category.
- Unlike Ordinal Encoding (which assumes an order), Label Encoding does **not assume any natural order** — it simply assigns integers **based on label appearance or alphabetically**.

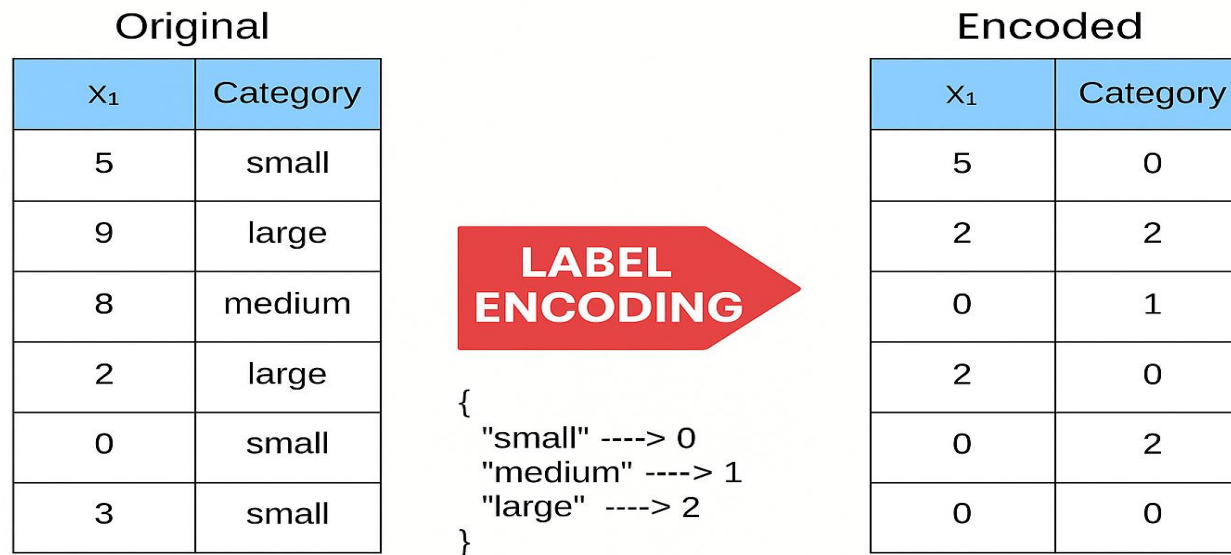


Fig : example of label encoding

LABEL ENCODING

```
from sklearn.preprocessing import LabelEncoder

# Initialize LabelEncoder
le = LabelEncoder()

# Apply Label Encoding
df['embarked_encoded'] = le.fit_transform(df['Embarked'])

# Show result
df[['Embarked', 'embarked_encoded']].sample(7)
```

	Embarked	embarked_encoded
245	Q	1
72	S	2
235	S	2
198	Q	1
36	C	0
490	S	2
770	S	2

DATA PREPARATION : MYTH Vs REALITY

● Myth	✓ Reality
One-time cleaning is enough	Data issues can reappear; cleaning is iterative
Just drop missing values	This can introduce bias; imputation or analysis is often better
All outliers should be removed	Some outliers are meaningful and should be kept
Feature scaling isn't important	Many models require it for proper functioning (e.g., SVM, KNN)
More features improve model performance	Irrelevant features can cause overfitting and reduce accuracy
Duplicates don't affect models	They can skew statistics and inflate sample size
Data prep is a one-time step	It must be repeated as data or requirements evolve
Modeling is more important than data prep	Bad data = bad model; prep is equally critical

THANK YOU